



UANL

UNIVERSIDAD AUTÓNOMA DE NUEVO LEÓN



FCFM

Análisis Numérico

M.A. Maria Del Carmen Martínez Cejudo

Proyecto 3 Interpolación Lineal

Sebastián Calderón Carrillo 2087472

Edson Armando Amaya Macias 2151976

Juan José Trujillo Garza 1972407

Grupo 031

Introducción

La interpolación lineal es una de las herramientas más intuitivas y fundamentales dentro del análisis numérico para estimar valores desconocidos que se encuentran entre dos puntos de datos discretos. Matemáticamente, se basa en la premisa de conectar dos coordenadas mediante una línea recta, asumiendo que el comportamiento de la función en ese pequeño intervalo es constante y predecible. Aunque en las sesiones teóricas suele resolverse de forma manual, el verdadero reto en ciencias computacionales radica en trasladar esta lógica a un entorno de programación funcional que contemple la interacción del usuario y prevenga excepciones matemáticas.

En este proyecto, se desarrolla una implementación interactiva en Python orientada a automatizar el cálculo tanto de la interpolación como de la extrapolación lineal. Más allá de simplemente codificar la fórmula de la pendiente, el objetivo de esta práctica es estructurar un script modular. El programa se encarga de interpretar expresiones matemáticas ingresadas en consola, evaluar dinámicamente los extremos del intervalo y calcular los márgenes de error absoluto y relativo frente al valor exacto. De este modo, la herramienta computacional no solo agiliza la obtención de resultados, sino que nos permite comprobar empíricamente cómo la exactitud de la aproximación numérica depende directamente de qué tan estrecho sea el intervalo de evaluación.

Codificación

En esta sección se presentan y explican las funciones implementadas en el código, destacando cómo cada una traduce las propiedades matemáticas del método de bisección y cómo se integran para formar el flujo completo del programa.

`interpolador_lineal(x, x0, x1, f_x0, f_x1)`

```
def interpolador_lineal(
    x: float, x0: float, x1: float, f_x0: float, f_x1: float
) -> float:
    """Aplica la interpolación lineal entre dos puntos conocidos.

    Args:
        x: Valor de x donde se desea estimar f(x).
        x0: Extremo izquierdo del intervalo.
        x1: Extremo derecho del intervalo.
        f_x0: Valor de la función en x0.
        f_x1: Valor de la función en x1.

    Returns:
        float: Valor aproximado de f(x).

    Raises:
        ValueError: Si x0 y x1 son iguales (división entre cero).
    """
    if x0 == x1:
        raise ValueError("Los puntos x0 y x1 deben ser distintos.")
    return f_x0 + ((f_x1 - f_x0) / (x1 - x0)) * (x - x0)
```

Explicación: Esta es la función principal que implementa el algoritmo matemático. Aplica la fórmula de interpolación lineal para encontrar un valor aproximado en el punto x basándose en dos puntos conocidos: (x_0, f_{x0}) y (x_1, f_{x1}) . Primero, cuenta con una validación de seguridad que lanza un error si x_0 es igual a x_1 , previniendo así una división por cero en la fórmula de la pendiente. Si los puntos son válidos, calcula y devuelve el valor aproximado sumando f_{x0} al producto de la pendiente $(f_{x1} - f_{x0}) / (x_1 - x_0)$ multiplicada por la diferencia $(x - x_0)$.

`_evaluar(funcion, x)`

```
def _evaluar(funcion: Funcion, x: float) -> float:
    """Evalúa f(x) traduciendo los errores comunes a mensajes claros."""
    try:
        return float(funcion(x))
    except ErrorFuncion:
        raise
    except ZeroDivisionError:
        raise ValueError(f"La función tiene una división entre cero en x = {x}.")
    except OverflowError:
        raise ValueError(f"La función desborda el rango de valores en x = {x}.")
    except ValueError as error:
        raise ValueError(f"La función no está definida en x = {x}: {error}")
```

Explicación: Esta función auxiliar actúa como intermediaria. Toma la función matemática introducida por el usuario y la evalúa numéricamente en el punto x especificado, manejando internamente los posibles errores matemáticos que puedan surgir en el cálculo (por ejemplo, evaluaciones en dominios inválidos) para devolver un resultado numérico en formato de punto flotante.

ejecutar_interpolacion()

```
def ejecutar_interpolacion() -> None:
    """Interactiva: lee f(x), x0, x1 y x, y muestra el valor interpolado."""
    print("\n--- Interpolación Lineal ---\n")
    print("Escribe f(x) usando la variable x.")
    print("Ejemplos:  x^2 - 3x + 2   |   ln(x) + 1   |   log(x)   |   exp(-x)")
    print("(Escribe la opción 2 del menú para ver toda la sintaxis)\n")

    expresion = input("Función f(x): ").strip()
    try:
        funcion = parsear_funcion(expresion)
    except ErrorFuncion as error:
        print(f"Error en la función: {error}")
        return

    x0 = leer_real("Extremo izquierdo x0: ")
    x1 = leer_real("Extremo derecho x1: ")

    if x0 >= x1:
        print("Error: x0 debe ser menor que x1 (izquierda antes que derecha).")
        return

    x = leer_real(f"Valor de x a interpolar (entre {x0} y {x1}): ")

    try:
        f_x0 = _evaluar(funcion, x0)
        f_x1 = _evaluar(funcion, x1)
        aproximado = interpolar_lineal(x, x0, x1, f_x0, f_x1)
    except ValueError as error:
        print(f"Error: {error}")
        return
```

```

pendiente = (f_x1 - f_x0) / (x1 - x0)

print("\n=== Resultados ===")
print(f"  f(x) = {expresion}")
print(f"  x0 = {x0}    f(x0) = {f_x0}")
print(f"  x1 = {x1}    f(x1) = {f_x1}")
print(
    f"  Pendiente m = (f(x1) - f(x0)) / (x1 - x0) "
    f"= ({f_x1} - {f_x0}) / ({x1} - {x0}) = {pendiente}"
)
print(f"\n  f({x}) ≈ {aproximado}")
print(f"  f({x}) ≈ {formatear_a_decimal(aproximado)} (6 decimales)")

try:
    exacto = _evaluar(funcion, x)
except ValueError:
    print("\n No fue posible evaluar f(x) en ese punto para comparar.")
    return

error_absoluto = abs(exacto - aproximado)
print(f"\n  Valor exacto f({x}) = {exacto}")
print(f"  Error absoluto = |exacto - aproximado| = {error_absoluto}")
if exacto != 0:
    print(f"  Error relativo = {error_absoluto / abs(exacto)}")
if x < x0 or x > x1:
    print(
        f"  Nota: x = {x} queda fuera del intervalo [{x0}, {x1}], "
        "por lo que se realizó extrapolación."
    )

```

Explicación: Esta función se encarga de toda la interacción con el usuario y orquesta el flujo de ejecución. Primero, solicita la función a evaluar $f(x)$ y los límites del intervalo (x_0 y x_1), validando de inmediato que el extremo izquierdo sea estrictamente menor que el derecho. Tras solicitar el punto x a interpolar, evalúa la función original en los límites y llama a `interpolar_lineal` para obtener el resultado. Finalmente, imprime un reporte en consola que desglosa el cálculo de la pendiente, el valor aproximado con formateo decimal, y los cálculos de error absoluto y relativo. Además, detecta si el punto x obligó a que matemáticamente se efectuara una extrapolación en lugar de una interpolación.

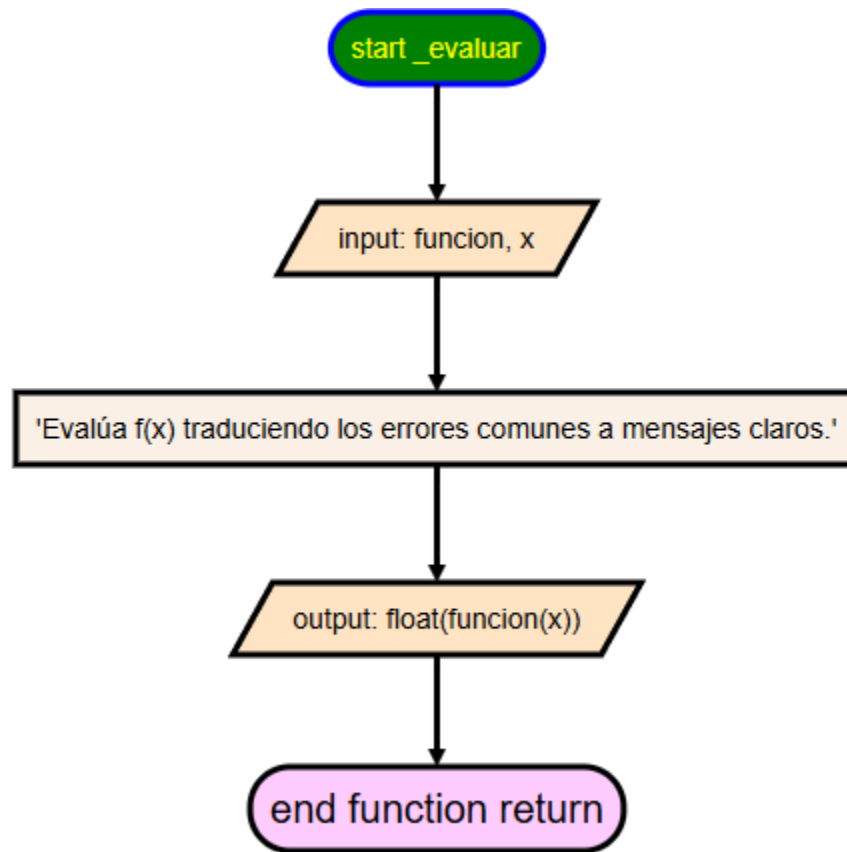
Módulo de Utilidades (utils.py) - Funciones Reutilizadas

Como parte de las buenas prácticas de diseño del proyecto, el módulo de utilidades incluye bloques de código que se fusionaron de entregas previas. Las siguientes son **funciones que se utilizaron en el "proyecto 2 Montante"**, implementadas para robustecer la lectura de datos interactiva:

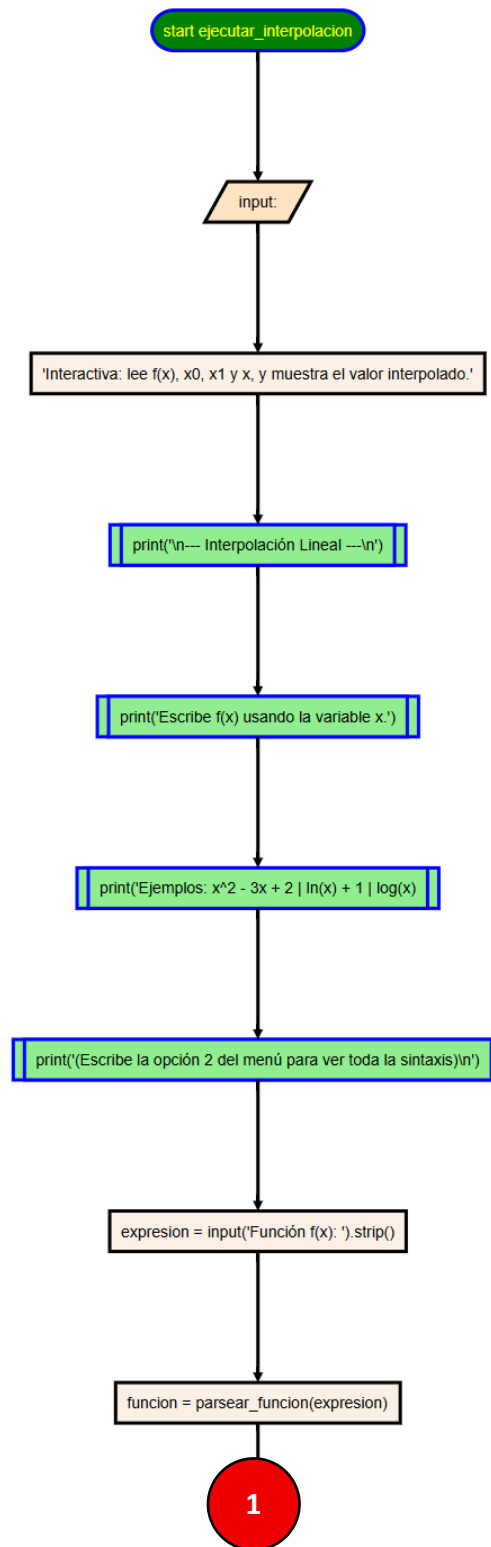
- **leer_real()**: Solicita y lee un número real (float) del usuario. Cuenta con un ciclo try-except que previene la caída del programa repitiendo la instrucción hasta recibir un valor numérico válido.
- **leer_opcion() / leer_entero()**: Funciones análogas que garantizan una correcta lectura de valores enteros, verificando que las selecciones no se salgan de rango durante los menús.
- **borrar_consola()** y **pausar()**: Herramientas encargadas de interactuar con el sistema operativo para limpiar la pantalla de la consola o pausar el flujo de texto (mediante Enter o pause), permitiendo al usuario una lectura cómoda de los resultados.

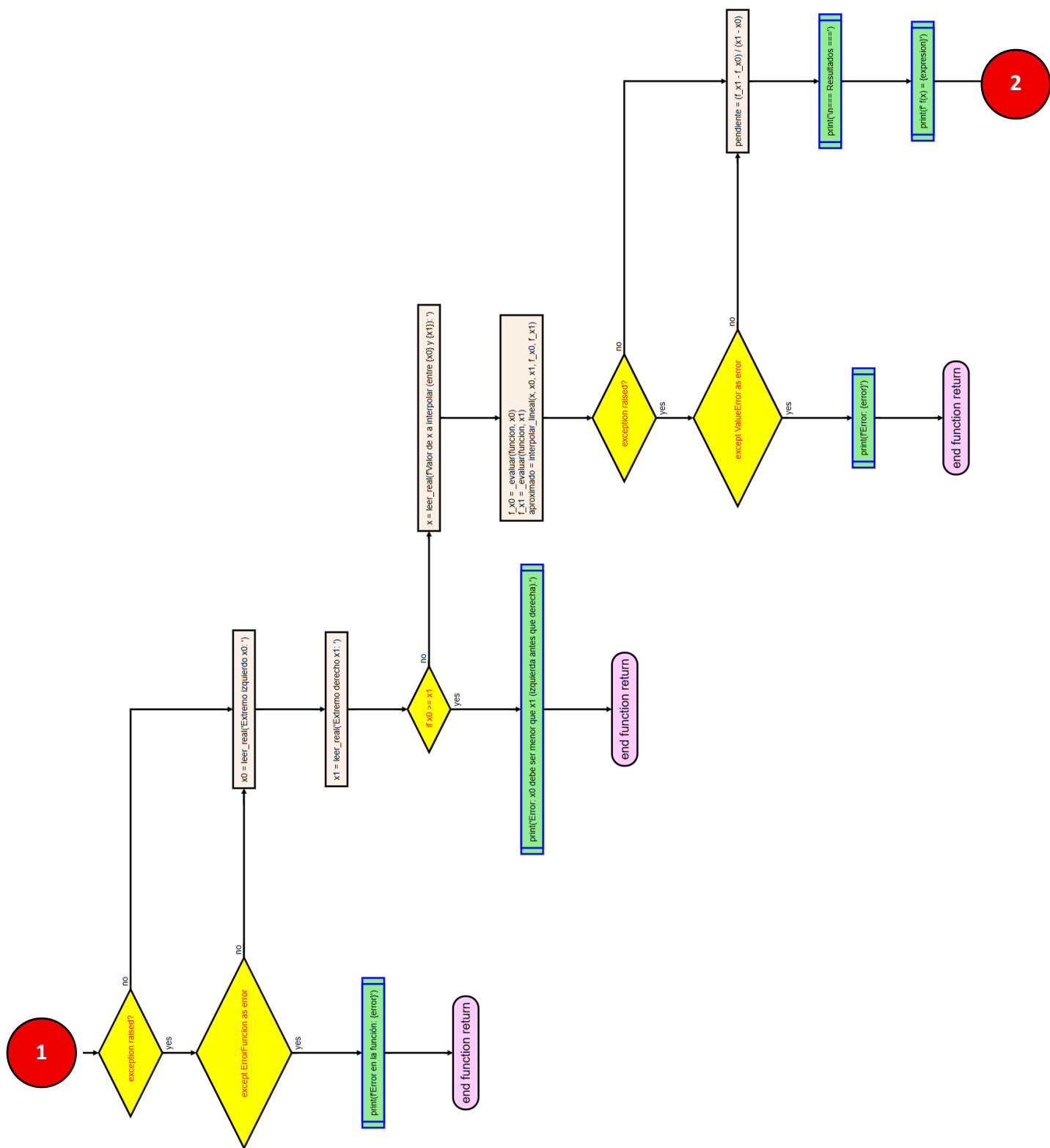
Diagramas de flujo

Evaluar Polinomio



Ejecución de interpolación





2

print(f' x0 = {x0} f(x0) = {f_x0}')

print(f' x1 = {x1} f(x1) = {f_x1}')

print(f' Pendiente m = (f(x1) - f(x0)) / (x1 - x0) = ((f_x1) - {f_x0}) / ({x1} - {x0}) = {pendiente}')

print(f'\n f(x) ~ {aproximado}')

print(f' f(x) ~ {formatear_a_decimal(aproximado)} (6 decimales)')

exacto = _evaluar(funcion, x)

exception raised?

except ValueError

print('\n No fue posible evaluar f(x) en ese punto para comparar.')

end function return

error_absoluto = abs(exacto - aproximado)

print(f'\n Valor exacto f(x) = {exacto}')

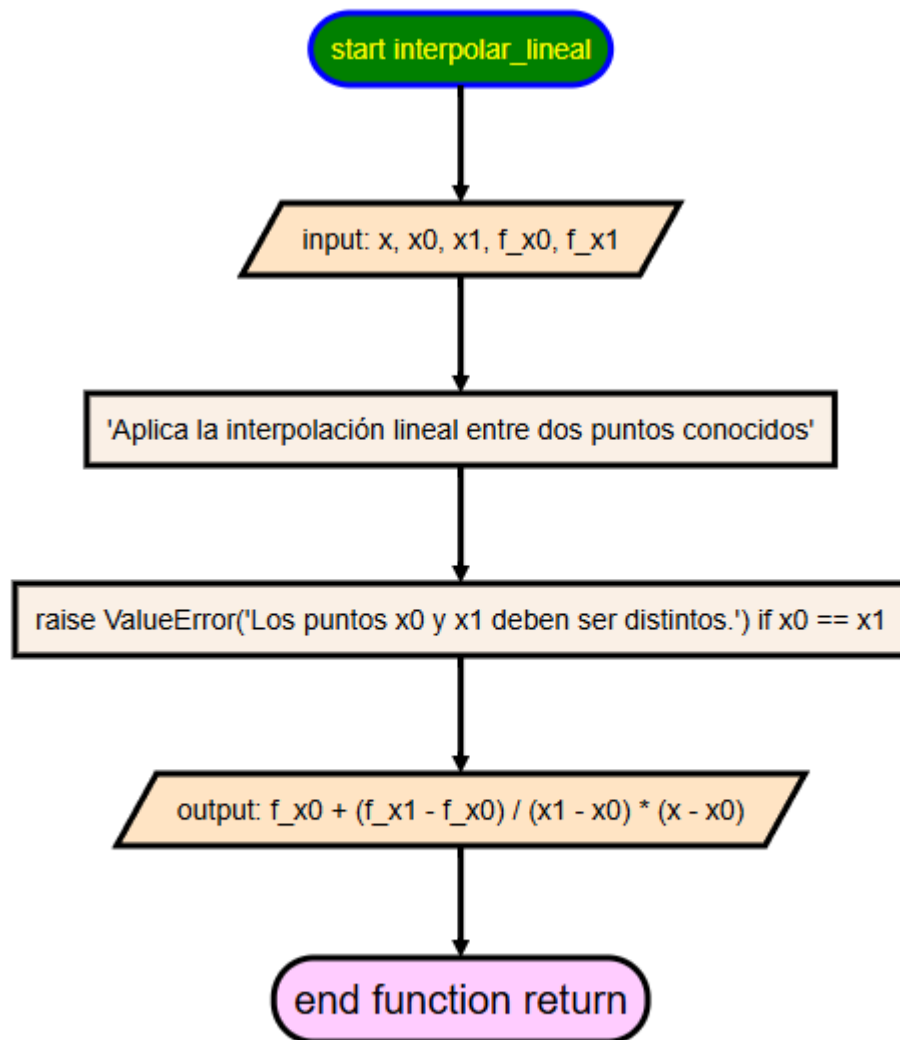
print(f' Error absoluto = {exacto - aproximado}')

print(f' Error relativo = {error_absoluto / abs(exacto)}' if exacto != 0)

print(f' Nota: x = {x} queda fuera del intervalo [{x0}, {x1}], por lo que se realizó extrapolación.' if x < x0 or x > x1)

end ejecutar_interpolacion

Interpolación de línea

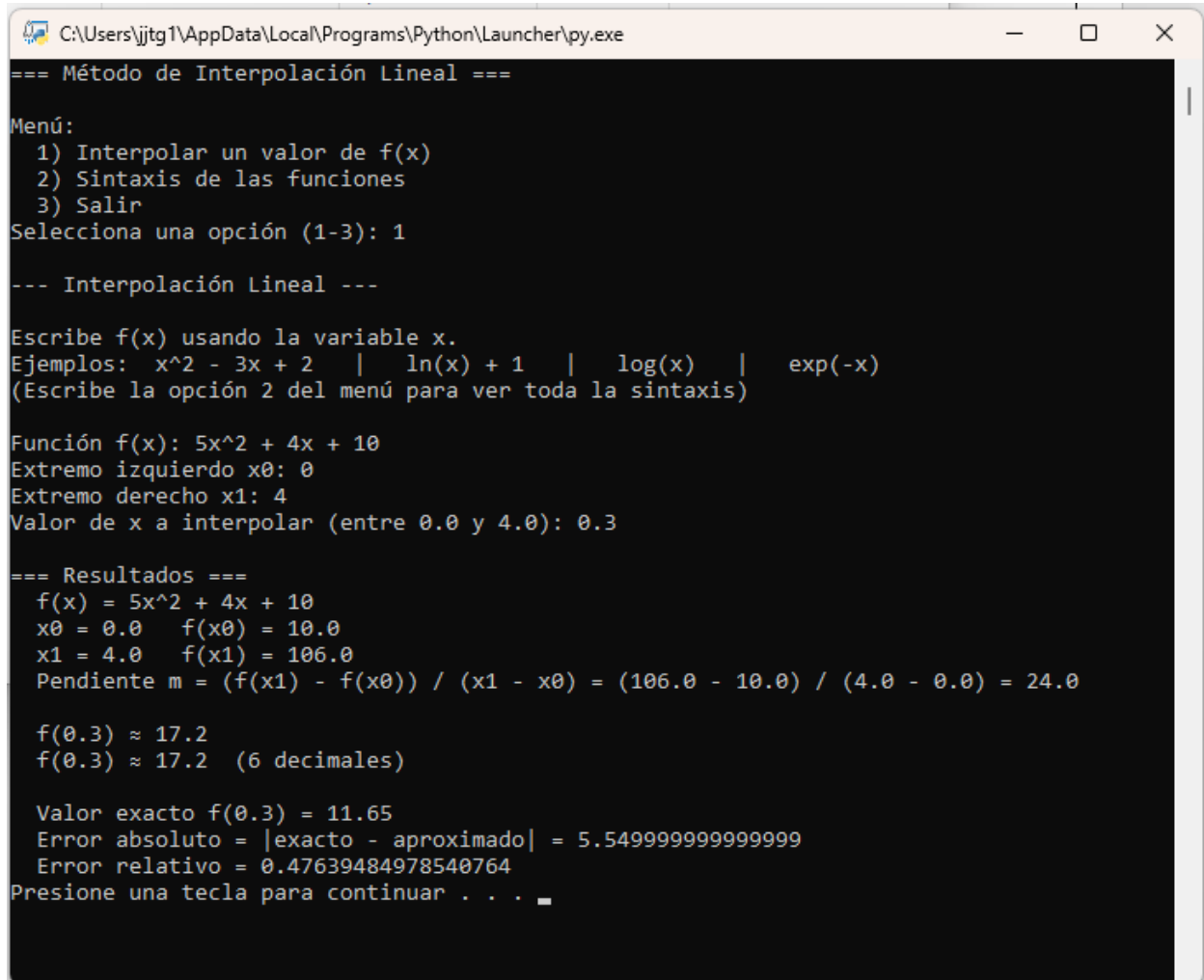


Conclusión general

El desarrollo de este proyecto nos permitió dar el salto de la teoría matemática a una implementación de software funcional, consolidando nuestra comprensión práctica de la interpolación lineal. En el aula es común ver estos métodos como meros procedimientos manuales, pero al llevarlos a código, nos enfrentamos a los verdaderos retos computacionales. Programar este algoritmo nos exigió pensar más allá de la fórmula y prever escenarios de falla, como evitar divisiones por cero si los puntos base ingresados son idénticos, además de diseñar validaciones que eviten que el programa colapse por errores de captura.

A través de la ejecución del script, pudimos analizar críticamente las fortalezas y debilidades de esta aproximación. Al programar el cálculo automático del error absoluto y relativo, comprobamos en tiempo real cómo la precisión del método está condicionada directamente por la distancia entre los puntos seleccionados y la curvatura de la función original. El hecho de que el código detecte y alerte cuando un valor cae en zona de extrapolación también nos ayudó a visualizar los límites de confianza empíricos de nuestros resultados.

Ejemplo de ejecución



```
C:\Users\jjtg1\AppData\Local\Programs\Python\Launcher\py.exe
=== Método de Interpolación Lineal ===

Menú:
  1) Interpoliar un valor de f(x)
  2) Sintaxis de las funciones
  3) Salir
Selecciona una opción (1-3): 1

--- Interpolación Lineal ---

Escribe f(x) usando la variable x.
Ejemplos: x^2 - 3x + 2 | ln(x) + 1 | log(x) | exp(-x)
(Escribe la opción 2 del menú para ver toda la sintaxis)

Función f(x): 5x^2 + 4x + 10
Extremo izquierdo x0: 0
Extremo derecho x1: 4
Valor de x a interpolar (entre 0.0 y 4.0): 0.3

=== Resultados ===
f(x) = 5x^2 + 4x + 10
x0 = 0.0   f(x0) = 10.0
x1 = 4.0   f(x1) = 106.0
Pendiente m = (f(x1) - f(x0)) / (x1 - x0) = (106.0 - 10.0) / (4.0 - 0.0) = 24.0

f(0.3) ≈ 17.2
f(0.3) ≈ 17.2 (6 decimales)

Valor exacto f(0.3) = 11.65
Error absoluto = |exacto - aproximado| = 5.549999999999999
Error relativo = 0.47639484978540764
Presione una tecla para continuar . . .
```